EPSR-test-ex30 makeato: one-atom .ato file (crashing Fortran)

Program(s): makeato | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises `makeato`, the EPSR setup tool [1] that creates a `.ato` file holding a single one-atom molecule from Lennard-Jones and charge parameters typed on stdin, the usual starting point for atomic liquids and ions in EPSR [2,3]. Both ways of giving the file name are tested: on the command line, and typed on stdin with an extension that is stripped.

Code exercised

Fortran (original, fortran_src/):

- `makeato.f`: `makeato` (prompts, atom label = first 3 characters of the name, box from density); `molecom_mod.f90`: `testsize_rotgrps`
- `ato_read_and_save.f`: `moloutc` (where the program crashes)
- `parse.f`: `parse`; `file_and_directory_commands.f`: `get_command_from_args`

Rust (port, rust/epsr-core/src/):

- `tools/makeato.rs`: `run`
- `tools/setup_common.rs`: `command_text`, `read_text`, `read_real5` (stdin answers)
- `ato.rs`: `moloutc` (check of the coarse-graining arrays, error return)

Physical system

Two single atoms. `Ow`: epsilon 0.65 kJ/mol, sigma 3.16 Å, mass 15.9994, charge -0.8476 e, symbol O, 300 K, 0.1 atoms/Å³ (close to the SPC/E water oxygen [4]). `Argon` (typed as `Argon.ato`): epsilon 0.97 kJ/mol, sigma 3.405 Å, mass 39.948, charge 0, symbol Ar, 85 K, 0.0213 atoms/Å³ (liquid-argon density). No simulation is performed; the program only writes the box description.

Inputs

Two stdin files written for this test: `makeato_Ow.stdin` (command `makeato Ow`, seven answers: epsilon, sigma, mass, charge, symbol, temperature, density) and `makeato_Ar.stdin` (command `makeato`, file name `Argon.ato` on the first line, then the same seven answers).

How the accuracy was verified

`commands.txt` (2 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.

The reference Fortran program crashes (SIGSEGV, exit 139) inside `moloutc`: `makeato` never calls `ctype`, so the coarse-graining arrays `natomsphere/pseudofacmol` are unallocated when `moloutc` reads them, and the buffered output is lost before it is flushed. Both expected outputs, `Ow.ato` and `Argon.ato`, are therefore empty (0 bytes in `fortran_reference.tar.gz`). The port reproduces this outcome: it writes the same empty files and exits with code 1 and the message `moloutc: natomsphere/pseudofacmol not allocated`. The test runner compares the files left behind, which are identical.

Notes

The case documents a bug of the original program rather than hiding it: a working `.ato` file is not produced by either implementation. Other cases (e.g. `makelattice`, ex37) build one-atom molecules through the `ctype` route instead.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] A. K. Soper, "Empirical potential Monte Carlo simulation of fluid structure", Chem. Phys. 202, 295-306 (1996).
[3] A. K. Soper, "Partial structure factors from disordered materials diffraction data: An approach using empirical potential structure refinement", Phys. Rev. B 72, 104204 (2005).
[4] H. J. C. Berendsen, J. R. Grigera and T. P. Straatsma, "The missing term in effective pair potentials", J. Phys. Chem. 91, 6269-6271 (1987).
[5] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.